SOFTWARE REQUIREMENTS SPECIFICATION

for

Natural Control

Version 1.0

Prepared by Group 7
Lucas DeBoer, Kevin Kim, Aaron Loh, Curtis Vos

Group 7, SFWRENG 4G06, McMaster University

November 3, 2026

Revision History

Table 1 summarizes the revision history.

Table 1: Revision history.

Date	Contributor	Change
2026-10-19	All	Completed for Revision 0 submission.

This document follows the requirements documentation structure presented in the Handbook of requirements and business analysis, by Bertrand Meyer, with the changes listed in the first chapter.

Contents

Changes from the Meyer template 7

Glossary 8

Goals 10
G.1 Context and overall objective . . . 10
G.2 Current situation . . . 10
G.3 Expected benefits . . . 10
G.4 Functionality overview . . . 10
G.5 High-level usage scenarios . . . 11
G.6 Limitations and exclusions . . . 11
G.7 Stakeholders and requirements sources . . . 11
G.8 User profiles . . . 13
G.9 Goals . . . 13

Environment 15
E.1 Glossary . . . 15
E.2 Components . . . 15
E.3 Constraints . . . 15
E.4 Assumptions . . . 15
E.5 Effects . . . 16
E.6 Invariants . . . 16
E.7 Standards and legal/regulatory factors . . . 16
Environment requirements . . . 16

System 17
S.1 Components . . . 17
S.2 Functionality . . . 19
S.2.1 Voice interface . . . 19
S.2.2 Task manager . . . 19
S.2.3 Action gate . . . 20
S.2.4 Outcome verification . . . 21
S.2.5 Action tools and integrations . . . 22
S.2.6 Accessible client and task history layer . . . 22
S.2.7 Safety requirements . . . 23
S.2.8 Non-functional requirements . . . 24
S.3 Interfaces . . . 25
S.4 Normal operation . . . 26
S.5 Undesired event handling . . . 26
S.6 Detailed usage scenarios . . . 29
UC1: Reply to an email . . . 29
UC2: Add an event from a webpage to the calendar . . . 30
UC3: Compare information on a website . . . 30
UC4: Correct or pause a running task . . . 31
UC5: Reconnect without duplicating an action . . . 32

S.7 Prioritization . . . 33
S.8 Verification and acceptance criteria . . . 33
S.9 Formalization: task state machine . . . 33

Project **36**
P.1 Roles and personnel . . . 36
P.2 Imposed technical choices . . . 36
P.3 Schedule and milestones . . . 36
P.4 Tasks and deliverables . . . 37
P.5 Required technology elements . . . 38
P.6 Risk and mitigation analysis . . . 38
P.7 Requirements process and report . . . 39

Requirement dependencies **40**

Likely and unlikely changes **41**

References **42**

Appendix: Interview transcript **43**

Appendix: AI use disclosure **46**

Appendix: Reflection **47**

List of Tables

1 Revision history. 2
2 Voice interface requirements 19
3 Task manager requirements 19
4 Action gate requirements 20
5 Outcome verification requirements 21
6 Action tools and integrations requirements 22
7 Accessible client and task history layer requirements 22
8 Non-functional requirements 24
9 Constants 25
10 Task state transitions 34
11 Tasks and deliverables 37
12 Goal-to-requirement traceability matrix 40

List of Figures

1 High-level activity diagram for the common usage pattern across Scenarios 1 to 3, showing the components (S.1) responsible for each step. 12
2 Component diagram. Green components are planned team work. Grey components are reused. White boxes are external services. 18
3 Normal operation: adding an event from a webpage to the calendar. 27
4 Pausing work while an approved action is in flight. 28
5 Task control states. Running groups the four running states in S.9. Grey states are final. External operations are tracked separately. 35

Changes from the Meyer template

- Glossary moved to the front, so every term is defined before first use.
- Added G.8 User profiles, to describe distinct user groups and their needs.
- Added G.9 Goals, copying the Problem Statement's goals so that the requirement traces resolve within this document.
- Added E.7 Standards and legal/regulatory factors, plus environment requirements.
- S.2 split into one subsection per component, with safety requirements ported from the Hazard Analysis.
- Added S.4 Normal operation, S.5 Undesired event handling, and S.9 Formalization (task state machine).
- Added requirement dependencies (traceability), likely and unlikely changes, and References.

Glossary

Project terms:

- User: the person who owns the connected accounts and gives commands. Other voices in the room are not users.
- Setup helper: a person who completes the initial assisted setup for the user.
- Session: the period during which the user is talking with the system, started and ended with the client's start control or a keyboard shortcut, or ended by disconnection. A session can contain several tasks.
- Task: one job the user asks for. It may pause before it is completed, cancelled, or failed.
- Consequential action: a write to user or account data, a submission, an external message, or a commitment such as accepting terms or granting access. Navigation and temporary preparation do not need approval unless they cause one of these effects. FR24 defines one exception for rejecting non-essential cookies.
- Approval: the user's explicit consent to one consequential action, or to a listed batch of similar actions, with their specific details; used once and void if any detail changes.
- Barge-in: the user speaking while the system is speaking; it stops the speech only.
- Stop request: an explicit "stop working" command or stop key. It pauses work and preserves the task. Cancellation ends the task.
- Pause acknowledged: the system received the stop request. Pause confirmed: no new task actions can start. Any earlier operation that is still pending is reported separately.
- Verified outcome: a result supported by a matching tool response, account record, or website confirmation. Unknown outcome: an action whose result cannot be determined.

Technical terms:

- LiveKit: an open-source platform for building real-time audio and video applications. [**?**]
- Hermes: an AI agent framework that plans multi-step tasks and calls external tools. [**?**]
- Browser Use: an open-source library that lets an AI agent control a web browser to complete tasks. [**?**]
- OAuth: an open standard for delegated authorization, letting a user grant an application limited access to their account without sharing a password. [**?**]
- VoiceOver: Apple's built-in screen reader for macOS and iOS. [**?**]
- STT (speech-to-text): technology that converts spoken audio into written text. [**?**]
- TTS (text-to-speech): technology that converts written text into spoken audio. [**?**]

- LLM (large language model): a machine learning model trained on large amounts of text to understand and generate natural language. Also is often refered to as simply an AI model.[**?**]
- POC (proof of concept): a small-scale demonstration built to test whether an idea is feasible before full development. [**?**]
- Composio: a hosted platform that connects AI agents to third-party applications through managed authentication and tool APIs. [**?**]
- CNIB (Canadian National Institute for the Blind): a Canadian non-profit organization providing programs and support for people who are blind, deafblind, or have low vision. [**?**]
- FMEA (Failure Mode and Effects Analysis): a systematic method for identifying potential failure modes in a system and analyzing their causes and effects. [**?**]
- JAWS: a screen reader for Windows that converts on-screen content to speech or braille. [**?**]
- ElevenLabs: a company providing AI-generated speech and audio, including text-to-speech and speech-to-text APIs. [**?**]

Goals

G.1 Context and overall objective

Many websites and applications communicate information, choices, and state changes visually. Blind users can face barriers completing multi-step digital tasks and knowing whether those tasks succeeded. Natural Control is a voice interface that lets a blind user delegate tasks in a web browser and in connected apps to an AI agent, while staying informed and in control.

G.2 Current situation

Today, blind users complete these tasks with screen readers such as VoiceOver, navigating each interface element by element, or with help from a sighted person. Existing AI computer-use agents assume the user can watch the screen to monitor and correct them.

In an interview with a former CNIB Deafblind Community Services (DBCS) staff member on October 17, 2026 (transcript in the appendix), the participant observed that screen readers are hard to learn for older, newly diagnosed users and often read irrelevant content, and that everyday tasks such as booking appointments often need a friend's help.

G.3 Expected benefits

- Delegate, don't navigate: the user simply states an outcome ("reply to Alex") instead of visually navigating the interface step by step. (G1)
- Control without sight: the user can understand, approve, correct, and stop the agent without seeing the screen. (G2, G4)
- Trustworthy results: results are checked against the real account or page before being reported, so the user does not need sighted help to confirm what happened. (G3) Correcting a wrong result after the fact is slow and costly for blind users [**?**].

G.4 Functionality overview

- **Delegated task execution:** carries out multi-step tasks in a web browser or a connected app from a single spoken goal, instead of the user navigating each step.
- **Clarification on demand:** asks for clarification when required details are missing or ambiguous rather than guessing about the user's intent.
- **Approval-gated actions:** requires explicit approval of exact details before completing consequential actions, such as sending, saving, or submitting.
- **Verified reporting:** checks task outcomes against the real account or page before reporting them as complete, rather than assuming a success after actions are taken.
- **Mid-task control:** accepts corrections, stop requests, barge-ins, and cancellation at any point during a task.

- **Accessible task history:** records what happens and makes the system's history reviewable by voice or through the keyboard and VoiceOver.

G.5 High-level usage scenarios

- **Scenario 1: Reply to an email.** A user asks the system to find a specific email and reply with a short message, approving the exact reply before it sends.
- **Scenario 2: Add a webpage event to Google Calendar.** A user asks the system to read an event off a webpage and add it to their calendar, hearing the details read back before it is created.
- **Scenario 3: Compare information on an unfamiliar website.** A sighted user who wants nonvisual interaction asks the system to compare different sets of prices for something on an unfamiliar website.
- **Scenario 4: Correct or stop a running task.** A user interrupts an in-progress task with a correction or an explicit stop request, and the system reports what has and has not completed.
- **Scenario 5: Reconnect after a dropped connection.** A user reconnects after losing their connection mid-task and resumes only after hearing the current state, without duplicating any action.

Figure 1 shows the common high-level flow behind Scenarios 1 to 3. Scenarios 4 and 5 are interruptions to this flow and are diagrammed separately in S.4 and S.5.

G.6 Limitations and exclusions

- Native Mac application control and general local-file tasks.
- Platforms other than Mac with Chrome; languages other than English.
- Phone calling and saved workflows (stretch goals).
- Background continuation: tasks always pause when the user disconnects; continuing unattended is a possible extension.
- Fully local model inference; the system uses cloud AI and speech providers.
- Users who cannot hear spoken output, including many deafblind users. Braille output through VoiceOver is a possible extension.

G.7 Stakeholders and requirements sources

- Primary: blind and visually impaired people who want to complete digital tasks without visual feedback. Initial evaluation: blind Mac users with screen-reader experience.
- Secondary: other people who want nonvisual interaction.
- Tertiary: website and application developers who affect the system's operational limits.
- Setup helper: a person who completes the assisted setup.
- Intervenors: support workers who may help deafblind users with setup and recovery.

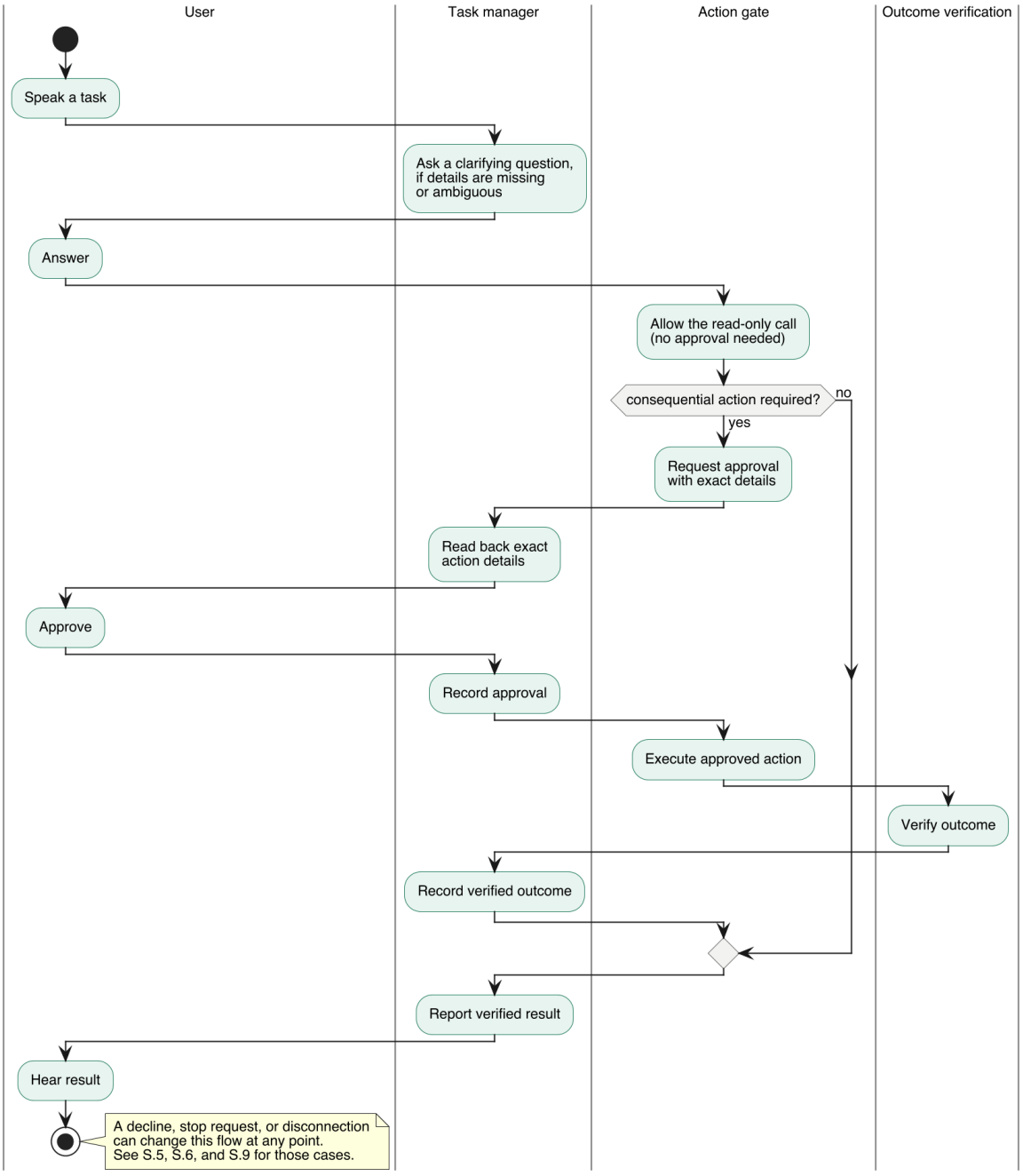


Figure 1: High-level activity diagram for the common usage pattern across Scenarios 1 to 3, showing the components (S.1) responsible for each step.

- Requirements sources: the Problem Statement, Development Plan, and Hazard Analysis documents; TA and instructor guidance; prior research (WebAIM survey [**?**], Morae [**?**], Azenkot and Lee [**?**]); an interview with a former CNIB Deafblind Community Services (DBCS) staff member on October 17, 2026 (transcript in the appendix); and planned participant sessions.

G.8 User profiles

The following is an expansion of the stakeholder groups who will actually use and benefit from our system. They are designed to demonstrate specific examples of who may be interacting with our system, and are intended to be refined after proper recruitment of participants has been completed.

- Experienced screen-reader user: fluent with VoiceOver or JAWS and keyboard navigation; wants speed on multi-step tasks and keeps their screen reader for everything else. Primary evaluation group.
- Less experienced or recently blind user: often older, with acquired vision loss, limited screen-reader skill, and possibly an older device; benefits most from delegation and clear spoken confirmation.
- Sighted user wanting nonvisual interaction: can see the screen but prefers or needs voice (busy hands, away from the screen); can verify visually, so has lower risk.
- Setup helper: comfortable with computers; connects accounts and grants permissions once, then is not needed for daily use.

G.9 Goals

The goals below are copied from Section 2 of the Problem Statement document. Their identifiers (G1 to G5) are the trace targets used in the requirement tables of S.2, and are distinct from the section numbers of this book (G.1 to G.9).

G1. Complete complex digital tasks independently. Within the initial browser and connected-account scope, users should be able to complete common, multi-step tasks without relying on visual feedback or sighted assistance. Success can be measured by task completion rate and the amount of sighted assistance that is required.

G2. Maintain awareness and control during digital tasks. Users should be able to understand relevant task progress and state changes, provide input when required, and stop or redirect tasks when necessary. Measure successful corrections, explicit stop acknowledgment and execution-stop confirmation, and approval before consequential actions. Clarification counts provide context; fewer questions alone do not establish better control.

G3. Accurately receive results and execute commands. Users should be able to have confidence that the program is correctly tracking the system's state, retrieving results, and executing their desired actions and commands. Compare reported outcomes and executed actions against independently verified application state and the user's request. Report incorrect or unsupported completion claims separately from successful actions.

G4. Recover from errors and unexpected outcomes. Users should be able to identify unexpected, misunderstood, or failed actions and recover without restarting the entire task or relying on sighted assistance. Measure whether recovery succeeds without sighted help or a full restart, along with recovery time and clarification turns.

G5. Provide a straightforward and reliable setup process. Individuals responsible for setting up the interface should have a smooth setup process. Routine use after initial setup should not require assistance from the setup helper. Success can be measured by the number of clicks or commands needed, the setup time before the program can be used, as well as how many times the setup helper is required to assist after installation.

Environment

E.1 Glossary

See the Glossary at the front of this document.

E.2 Components

This section contains a list of elements of the environment that may affect or be affected by the system, including other systems the system must interface with.

- Users: the person giving spoken commands and acting on the system's results; see the Glossary and G.7–G.8 for roles and profiles.
- Laptop hardware: the user's Mac, including its microphone and speakers.
- Platform: macOS with Google Chrome, and the user's existing VoiceOver screen reader.
- Network: required for cloud speech, AI, and connected apps' API traffic.
- AI model and speech providers: cloud services for language understanding, speech-to-text, and text-to-speech.
- Websites the user asks about (unmodified, not under our control).
- External accounts and services.
- Composio cloud: an external hosted service that stores OAuth tokens for our connected apps. Its "Don't store data" setting is turned on so tool inputs and outputs are not logged. Our own code calls it through the Composio SDK (S.1).

E.3 Constraints

The following is a list of obligations and limits imposed by the environment.

- Mac with Google Chrome, in English.
- Provider terms of use and data retention policies apply to anything sent to them.
- A hosted backend cannot reach the user's Mac or its signed-in browser sessions, so controlling the user's own browser requires a component running locally.

E.4 Assumptions

- A setup helper completed setup correctly (accounts connected, permissions granted).
- Only the user gives commands; other voices are noise.
- Connected accounts belong to the user; development and evaluation use test accounts.

- The user can hear spoken output (see G.6).

Speech recognition, the AI model, websites, and external services are not assumed to work correctly; their failures are covered by the Hazard Analysis and S.5.

E.5 Effects

- Users will rely less on outside help to complete computer tasks, as the system will assist in their completion.
- Desired properties in connected apps are affected (e.g. read, created, updated, or deleted).
- Website forms may be filled and submitted by the system, subject to user approval.
- Input data can reach AI and speech vendors.
- Content and progress is processed and reported back to the user through the computer's speaker hardware.

E.6 Invariants

- External account and page states can change outside the system at any time, through other devices, other users, or the provider itself; the system's last known state should not be assumed current.
- A delivered message or submitted form cannot be withdrawn by the system after the fact; only the provider or recipient can undo it, if at all.
- Authentication codes (and credentials from services like OAuth) are controlled by the provider: they expire, can be revoked, and may require re-authorization independent of the system's own state.
- The user's existing accessibility tools (VoiceOver, keyboard navigation) keep working while the system runs.

E.7 Standards and legal/regulatory factors

- WCAG 2.2 Level AA: accessibility standard for the web client, adopted voluntarily [**?**].
- AODA (Accessibility for Ontarians with Disabilities Act): Ontario accessibility law referencing WCAG; its compliance obligations are not required as they apply to organizations with Ontario employees (20+ for reporting, 50+ for website compliance), but it is a relevant reference for our team [**?**].
- Open-source licenses: our code uses MIT licensing; reused components keep their own licenses and notices.
- Coding standards: PEP 8 for Python; strict TypeScript with a shared linter for the client.

Environment requirements

- NFR.E.1: The system shall run in English on macOS with Google Chrome.
- NFR.E.2: The web client shall meet WCAG 2.2 Level AA.
- NFR.E.3: Python code shall pass the team's PEP 8 linter checks; TypeScript code shall pass strict type checks and the shared linter.

System

S.1 Components

The proposed system has eight components. These match the Hazard Analysis. Figure 2 shows their connections and external services. Team-owned components are planned work. The POC must test the integration boundaries.

- **Accessible client:** the web interface for sessions and task history. It provides keyboard controls and works alongside VoiceOver.
- **Voice interface:** uses LiveKit for audio and conversational turn handling. Our integration passes requests and corrections to the task manager. It speaks relevant questions and results.
- **Task manager:** starts tasks and tracks their state. It handles corrections, pauses, cancellation, and resumption (S.9).
- **AI agent:** Hermes plans tasks and proposes tool calls. Each call must pass through the action gate.
- **Action gate:** checks every proposed action. It requests approval for writes and consequential commitments. Approval details come from the proposed call. The gate also blocks new task actions while work is paused.
- **Outcome verification:** checks evidence for each consequential action. Evidence may come from its tool response or a targeted account or page check. The result is recorded as verified, partial, failed, or unknown.
- **Action tools and integrations:** Browser Use handles websites. Connected app tools use the Composio SDK. The proposed integration places approval checks before execution and outcome checks afterward. The POC must verify that all action paths pass through these checks. Credential handling also needs verification.
- **Task history layer:** records task state and action outcomes. It also records approvals and action IDs. These records support accessible review and recovery after disconnection.

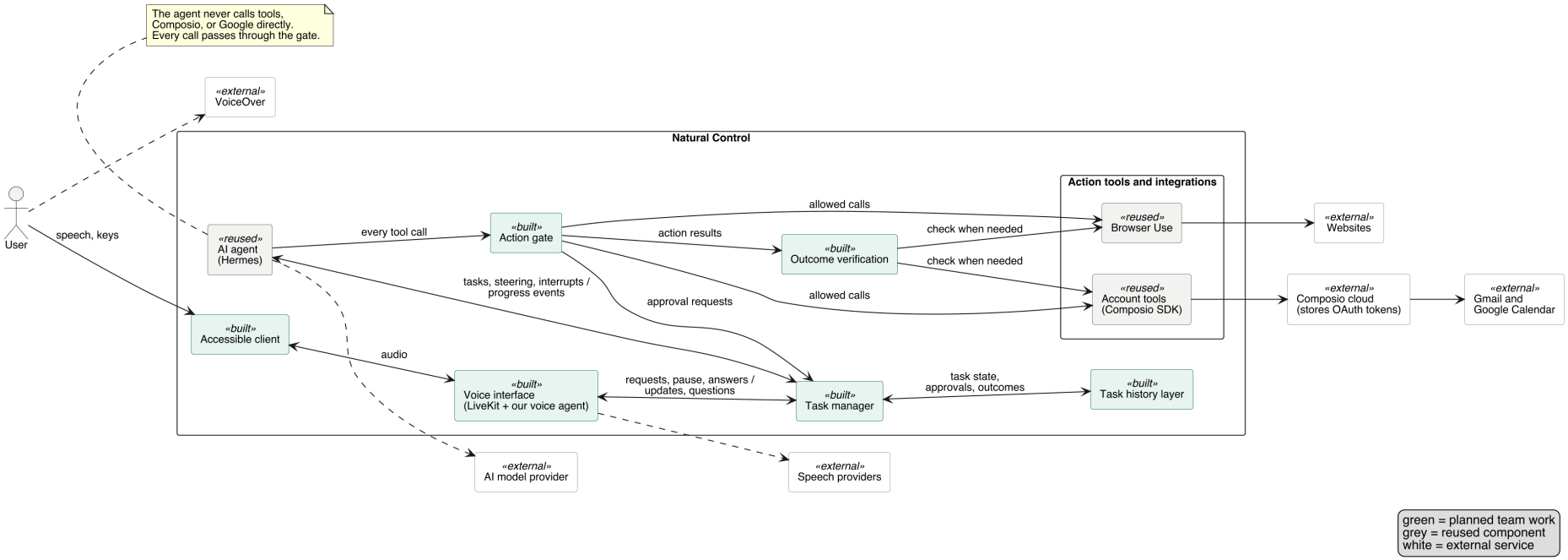


Figure 2: Component diagram. Green components are planned team work. Grey components are reused. White boxes are external services.

S.2 Functionality

Requirements are grouped by their responsible component (S.1). Each has a rationale and trace links. G1 to G5 refer to project goals. FR.Sa and NFR.Sa refer to safety requirements. The task manager and action gate constrain the AI agent. Outcome verification checks its results.

S.2.1 Voice interface

Table 2: Voice interface requirements

ID	Requirement	Rationale	Traces to
FR1	The system shall accept spoken input during a session. It shall support pauses within a thought through the selected voice framework's turn handling.	Voice is the main input for everyday use. Turn timing needs POC and user evaluation.	G1
FR2	The user shall be able to interrupt spoken output. This shall not restart the task or pause its execution by itself. A correction or explicit work-stop request shall follow its own handling.	Users need to cut off long speech without losing work.	G2
FR3	An explicit "stop working" command or the stop key shall pause the task (FR8). A bare "stop" shall stop speech and pause work. The system shall clarify what the user wants next. An explicit request to cancel or abandon the task shall end it.	Speech interruption, work pause, and cancellation have different effects. The key provides a backup when voice fails.	G2; FR.Sa.1, FR.Sa.4
FR16	The system shall let the user correct part of a read-back (for example, one sentence or one detail) by voice without restarting the request.	Interview: fixing a read-back usually means starting over. Correcting speech input is slow for blind users [**?**].	G4
FR18	Spoken updates and summaries shall include only information relevant to the current task, with more detail available on request.	Interview and WebAIM survey [**?**]: screen readers read too much irrelevant content.	G2

S.2.2 Task manager

Table 3: Task manager requirements

ID	Requirement	Rationale	Traces to
FR4	The system shall ask when required details are missing or ambiguous. It shall recheck relevant information before continuing.	Guessing risks acting on the wrong item.	G3; FR.Sa.9
FR5	The system shall report meaningful progress and blockers. It shall answer requests for current status. Routine updates shall follow the task and conversation without exceeding the maximum silence interval in NFR.Sa.3. Pending questions and approvals shall not expire while the user remains connected.	Users need awareness without constant narration. Silence is not approval.	G2; NFR.Sa.3
FR6	The system shall accept corrections during a task without requiring a restart.	Restarting a long task by voice is costly.	G4

ID	Requirement	Rationale	Traces to
FR7	Disconnection shall pause the task as in FR8. On reconnection the system shall report its state and reconcile uncertain outcomes. It shall wait for a clear request to resume work.	Reconnecting the voice session does not authorize more actions.	G2, G4; FR.Sa.7
FR8	A stop request shall block new task actions and preserve task context. The system shall stop or suspend ongoing work where safe and supported. An operation already submitted may settle if it cannot be stopped safely. The system shall distinguish acknowledgment from confirmed pause. It shall report completed work and unfinished work. It shall also report any outcome still unknown. Checks during a pause shall only reconcile earlier actions. Work shall resume only on a clear user request.	Pausing does not undo a submitted action. The user needs an accurate account of what remains.	G2, G4; FR.Sa.5.i, FR.Sa.5.ii, FR.Sa.5.iii
FR19	A new request during an active task shall be treated as a correction when that is clear. Otherwise the system shall ask whether to queue it or replace the current task. Replacement shall end the earlier task before the new one starts.	One active task keeps spoken progress easy to follow (S.9, P6).	G2
FR20	When the user moves to another request the system shall withdraw pending questions and approvals. It shall state that the pending action was not executed. It shall distinguish this from earlier completed work. The new request shall follow FR19.	An abandoned approval must not be used later.	G2, G3; FR.Sa.15
FR21	At session start the system shall announce paused tasks and their last known state. The user may resume or cancel. Before resuming the system shall check relevant state and reconcile earlier actions. Pending approvals shall be requested again. Reconnection alone shall not resume work.	Recovery must preserve context without repeating actions.	G2, G4; FR.Sa.8

S.2.3 Action gate

Table 4: Action gate requirements

ID	Requirement	Rationale	Traces to
FR9	Writes and consequential commitments shall require approval of their exact details. These include sending, saving, deleting, submitting, granting access, and accepting terms. Approval shall be an explicit affirmative answer to the pending approval question. Silence is not approval. Changed details require new approval. FR24 defines the cookie-rejection exception.	Users retain control over changes and commitments. Approval checks use the proposed tool call (ADR 0001).	G2, G3; FR.Sa.3, FR.Sa.13, FR.Sa.14

ID	Requirement	Rationale	Traces to
FR10	Reading, navigation, and temporary preparation shall proceed without approval when they cause no consequential effect. Autosaved drafts are writes. A click that submits data is consequential. Unknown effects shall require approval. A failed read-only lookup may be retried once before it is reported as a blocker.	Classify the effect of an action rather than the name of its tool.	G1, G2; FR.Sa.13
FR11	Each approved action shall have a unique ID. A completed action shall not be repeated as a retry. Before retrying an uncertain action the system shall check its outcome. If it remains unknown the system shall ask the user. A new attempt requires fresh approval.	An uncertain result must not cause a duplicate write.	G3; FR.Sa.6, FR.Sa.18
FR13	The system shall treat webpage and email content as data, never as instructions.	Websites and emails can contain text written to manipulate agents.	G3; FR.Sa.16

Approval details. Similar actions may share one approval. The read-back shall state the number of actions, the intended changes, and a concise description identifying the affected items. Full item details shall be available on request. The approved batch shall retain each item's identity and exact details. Each item may run once. Partial failures shall be reported per item. A pause shall block remaining items and withdraw their unused approvals.

For email the read-back shall name the recipients and subject. It shall read the body in full up to SHORT_MESSAGE_LENGTH. For longer messages it shall give a summary and offer the full text. The user may request a sentence-by-sentence read-back. A reduced-prompt permission mode is deferred.

S.2.4 Outcome verification

Table 5: Outcome verification requirements

ID	Requirement	Rationale	Traces to
FR12	Each consequential result shall be reported as verified, partial, failed, or unknown. Evidence shall match the intended action and details. It may be a completion response from the connector, a matching account record, or a website confirmation. Another check is required when that evidence is unclear. A queued request shall not be reported as delivered.	The reported result must match the available evidence. A second lookup is not always needed.	G3; FR.Sa.10

S.2.5 Action tools and integrations

Table 6: Action tools and integrations requirements

ID	Requirement	Rationale	Traces to
FR14	The system shall request only the account permissions it needs, and explain them during setup, including which providers receive which data.	Narrow permissions limit the damage any other failure can do.	G5; FR.Sa.17
FR17	The user shall be able to restrict access to specific accounts, senders, folders, or resources. The system shall not access or act on restricted resources.	The former CNIB staff interview raised concerns about sensitive account data.	G2
FR23	Website passwords shall be entered through a private setup page and stored encrypted. They shall not be spoken or placed in the conversation. Saved credentials shall be used only for their intended site. The user may speak a one-time authentication code or approve a push on their device. The system shall explain that spoken codes pass through the configured speech and conversation services. A site that requires a device-bound passkey shall use a manual handoff.	Spoken codes can be heard nearby and processed by providers. Private typed code input is a proposed alternative.	G2, G5; FR.Sa.17
FR24	The system may automatically choose a clearly identified "reject non-essential cookies" or "necessary only" option. Any ambiguous consent choice shall require explicit user approval. This is a narrow exception to FR9. Accepting cookies or terms shall require approval.	A limited exception reduces repeated consent prompts. It does not authorize accepting terms.	G2, G3; FR.Sa.13

S.2.6 Accessible client and task history layer

Table 7: Accessible client and task history layer requirements

ID	Requirement	Rationale	Traces to
FR15	The system shall provide accessible task history and spoken summaries from recorded events. The user shall be able to delete that history.	Users need to review what happened without sight.	G2; FR.Sa.2
FR22	The client shall provide controls to start and end a voice session. A keyboard shortcut shall also be available. Ending a session shall pause active work as in FR8.	Deliberate activation avoids an always-on microphone.	G2, G5

S.2.7 Safety requirements

The following safety requirements are based on the Hazard Analysis. Hazard IDs refer to its FMEA rows.

Functional safety requirements

- **FR.Sa.1:** The system shall provide a keyboard stop control that works whenever the client is running and does not conflict with VoiceOver commands. (H1-a)
- **FR.Sa.2:** The system shall build the task history only from recorded actions and verified outcomes, labelling unknown entries. (H8-a)
- **FR.Sa.3:** Before a consequential action the system shall read back its key details. It shall proceed only after explicit approval. The cookie-rejection exception is defined in FR.Sa.13. (H2-a)
- **FR.Sa.4:** Explicit stop phrases and the stop key shall use a fixed control path. They shall not depend on the planning agent choosing to pause. (H2-b)
- **FR.Sa.5.i:** After a stop-work request, the system shall pause the task and block all further actions for it. Only checks of earlier outcomes may continue running. (H2-c, H3-a)
- **FR.Sa.5.ii:** After a stop-work request, the system shall immediately acknowledge and confirm completion of the request as separate messages before reporting completed work, unfinished work, and uncertain outcomes. (H3-a)
- **FR.Sa.5.iii:** When a task is paused, an explicit user request shall resume or terminate the task. If terminated, earlier actions are not undone. (H3-a)
- **FR.Sa.6:** Each approved action shall have a unique ID. An action with that ID shall not execute twice. (H3-b)
- **FR.Sa.7:** Disconnection shall block new task actions and pause the task. Reconnection alone shall not resume it. (H3-b)
- **FR.Sa.8:** Task state and approval records shall survive restart. The system shall announce interrupted tasks with their last known state. Unused approvals shall be withdrawn. (H8-b)
- **FR.Sa.9:** The system shall ask a clarifying question when details required for an action are missing or ambiguous. (H4-a)
- **FR.Sa.10:** The system shall report a task as verified, failed, or partial only after verifying its outcome with evidence that matches the intended action and its details, such as a connector result, an account record, or a website confirmation. If the evidence is unclear, the system shall check again or report the outcome as unknown. (H4-b, H6-a, H6-b, H7-a)
- **FR.Sa.11:** A configured step or time limit shall pause the task and prompt the user. Time spent waiting for user clarification or approval shall not count toward the execution time limit. (H4-c)
- **FR.Sa.12:** An external service failure shall pause work and the system shall follow the steps outlined in FR.Sa.5.i, FR.Sa.5.ii, and FR.Sa.5.iii. (H4-d)
- **FR.Sa.13:** Unclear actions and consequential actions such as submissions, external messages, saved data changes, or granting permission require explicit approval before being executed; only actions with no such lasting effect do not require approval. One exception to this rule: rejecting non-essential cookies or only allowing necessary cookies on webpages does not require explicit approval. (H5-a)

- **FR.Sa.14:** The system shall bind each approval to the action's exact details, use it at most once, and require new approval if any detail changes. A pause, disconnection, cancellation, or task replacement shall withdraw unused approvals. (H5-b, H7-a)
- **FR.Sa.15:** When a pending approval expires or is withdrawn, the system shall tell the user that the action was not executed and shall state which earlier actions were completed. (H5-c)
- **FR.Sa.16:** The system shall treat webpage and email content as data, and shall only perform consequential actions that trace to the user's request. (H7-b)
- **FR.Sa.17:** The system shall request only the account permissions each tool needs, and shall store and use passwords and other credentials securely, keeping them out of source control, logs, and the AI model input. (H7-c)
- **FR.Sa.18:** The system shall check the service or item state directly and confirm the user's approval before retrying an action whose outcome is unknown. (H7-d)
- **FR.Sa.19:** Before deletions, the system shall read back the item's identifying details, state whether the operation is reversible, and use recoverable deletion where supported. (H5-d)
- **FR.Sa.20:** If voice input, voice output, or speech services become unavailable, the system shall stop as outlined in FR.Sa.5.i, accept no new tasks requests until voice functionality is restored, and shall not resume unless user explicitly requests it. (H2-c)

Non-functional safety requirements

- **NFR.Sa.1:** The system shall audibly acknowledge a stop request within STOP_ACK_TIME of the end of the utterance or key press. (H2-b)
- **NFR.Sa.2:** The client shall signal disconnection within CONNECTION_NOTICE_TIME. A local sound shall be used if network speech is unavailable. (H3-b)
- **NFR.Sa.3:** While a task is running, the system shall give a spoken status update after IDLE_PROMPT_TIME without spoken progress, outlining progress made or blockers. (H4-c)

S.2.8 Non-functional requirements

Table 8: Non-functional requirements

ID	Requirement	Rationale	Traces to
NFR1	Session controls, work pause, approval, and task history shall work with VoiceOver and the keyboard alone.	Users rely on their existing assistive tools.	G2, G5
NFR2	At least TARGET_COMPLETION of tasks in the declared task suite shall be completed correctly without sighted help. The task suite shall include representative connected apps as well as unfamiliar website tasks.	Measures the main benefit: independent task completion.	G1
NFR3	A work-stop request shall be acknowledged within STOP_ACK_TIME. Time to block new actions and time to reconcile in-flight work shall be measured separately.	A quick acknowledgment does not prove earlier actions ended.	G2; NFR.Sa.1

ID	Requirement	Rationale	Traces to
NFR4	The system shall begin its response within RESPONSE_TIME after the selected turn detector commits an ordinary request. Premature replies and missed interruptions shall also be evaluated.	Fast replies must not cut off unfinished thoughts.	G2
NFR5	The application shall not retain raw microphone audio. It shall exclude credentials and authentication codes from task history, logs, and source control. Setup shall explain provider processing and retention separately.	Application storage rules do not establish provider privacy guarantees.	G5; FR.Sa.17

Constants below remain provisional. The POC will inform timing choices. Routine speech has no fixed update interval.

Table 9: Constants

Constant	Meaning	Value
STOP_ACK_TIME	Time to audibly acknowledge a work-stop request	TBD after POC (Development Plan provisional: 2 s)
RESPONSE_TIME	Time from committed user turn to response start	TBD after POC
CONNECTION_-NOTICE_TIME	Time to signal connection loss (NFR.Sa.2)	TBD after POC
IDLE_PROMPT_TIME	Maximum interval without spoken progress during active task execution	TBD after POC
SHORT_MESSAGE_-LENGTH	Longest email body read in full before approval	TBD after POC
TARGET_COMPLETION	Share of tasks completed correctly without sighted help	TBD (Development Plan provisional: 80%)

S.3 Interfaces

- **User interface:** a web client in Chrome. It provides session controls and task status. The user can pause, resume, or cancel work. Approval details and history are accessible through VoiceOver and the keyboard. Spoken questions can be repeated on request. The system shall explain the keyboard shortcut for stopping work during setup and repeat the shortcut on request. The shortcut shall not conflict with VoiceOver controls.
- **Program interfaces:** LiveKit carries audio and speech events. The Hermes integration carries task requests, corrections, and progress. The task manager controls execution separately from speech. Browser Use and Composio supply action tools. Each tool call passes through the gate.

S.4 Normal operation

Figure 3 follows one request from start to finish: the user asks to add an event found on a webpage to their calendar.

1. The user starts a session from the client (FR22) and speaks the request. The voice interface passes it to the task manager. It records the task and starts the agent in Working. The user hears a short acknowledgment.
2. The agent reads the page. The action gate lets this read-only call through to Browser Use and screens the returned content for injected instructions (FR10, FR13).
3. The agent proposes event creation. The gate holds the write and requests approval of its exact details. The task enters AwaitingApproval.
4. The voice interface reads the details back. The user says "yes." The task manager records approval and releases the call. The task enters Executing (FR9).
5. The gate submits the call with a unique action ID (FR11). The returned event details confirm creation. Outcome verification checks those details and reports the result (FR12). A separate lookup is used if the response is unclear.
6. The agent finishes the task (state Done). The task history records each step (FR15), and the user hears the verified result.

S.5 Undesired event handling

- **Misheard or ambiguous request:** ask for clarification. Read back key details before approval. (H2-a, H4-a)
- **Missed work-stop request:** provide a keyboard backup. Block new task actions once the request is recognized. (H2-b, H3-a)
- **Disconnection:** pause work. Report the state after reconnection and wait for an instruction to resume. (H3-b)
- **Provider failure:** pause work. Explain what completed and what remains unfinished or unknown. (H4-d)
- **Unknown outcome:** check the affected account or page before any retry. Ask the user if the outcome remains unknown. (H7-d)
- **Injected instructions:** treat webpage and email text as data. Do not let it authorize actions. (H7-b)

Figure 4 shows a work-stop request during event creation. The system acknowledges it within STOP_ACK_TIME. It blocks new task actions and pauses the task. An earlier submission may still complete. The system reports its result or marks it unknown. A pause does not promise that the external service stopped processing the submission (FR8).

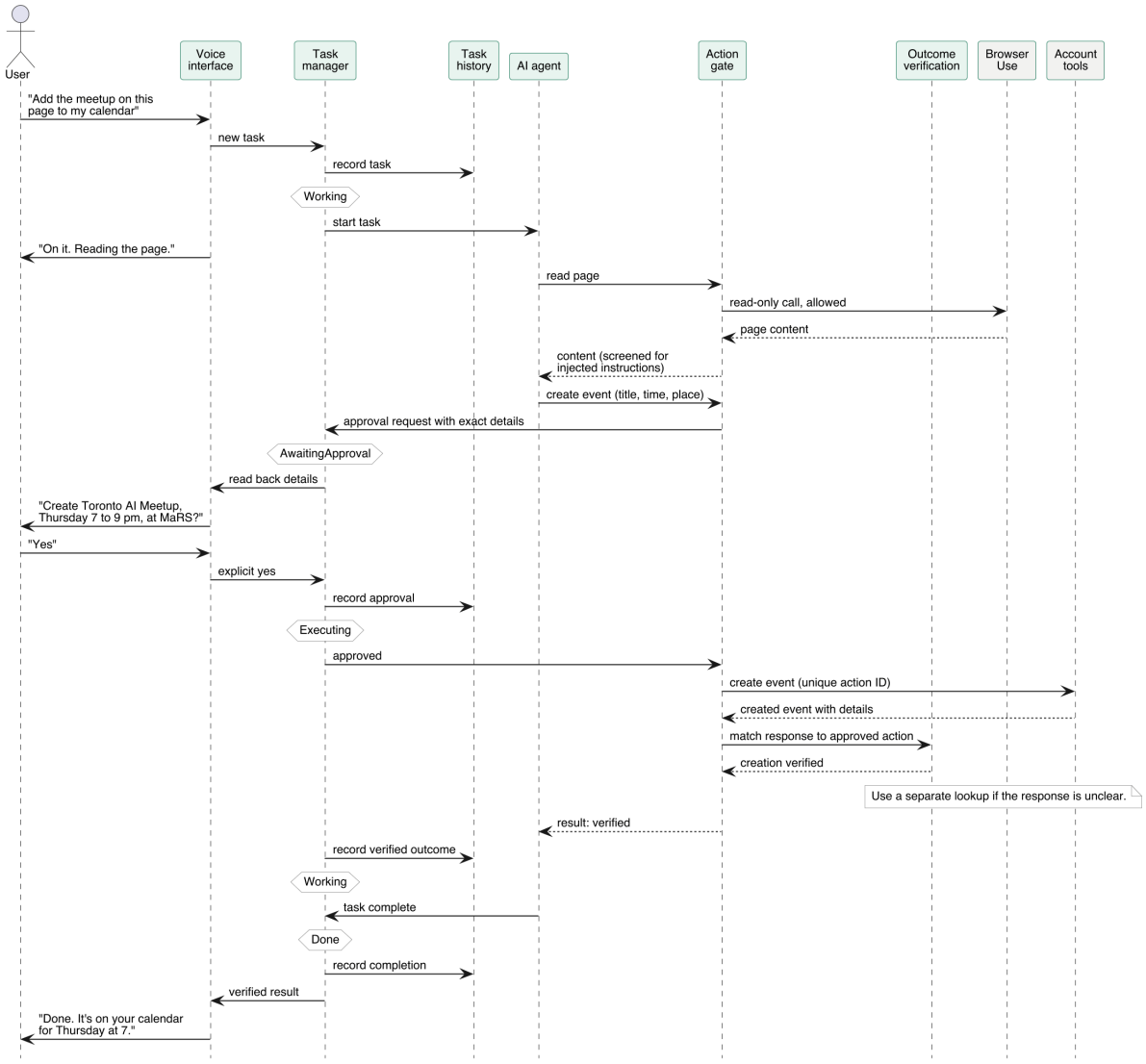


Figure 3: Normal operation: adding an event from a webpage to the calendar.

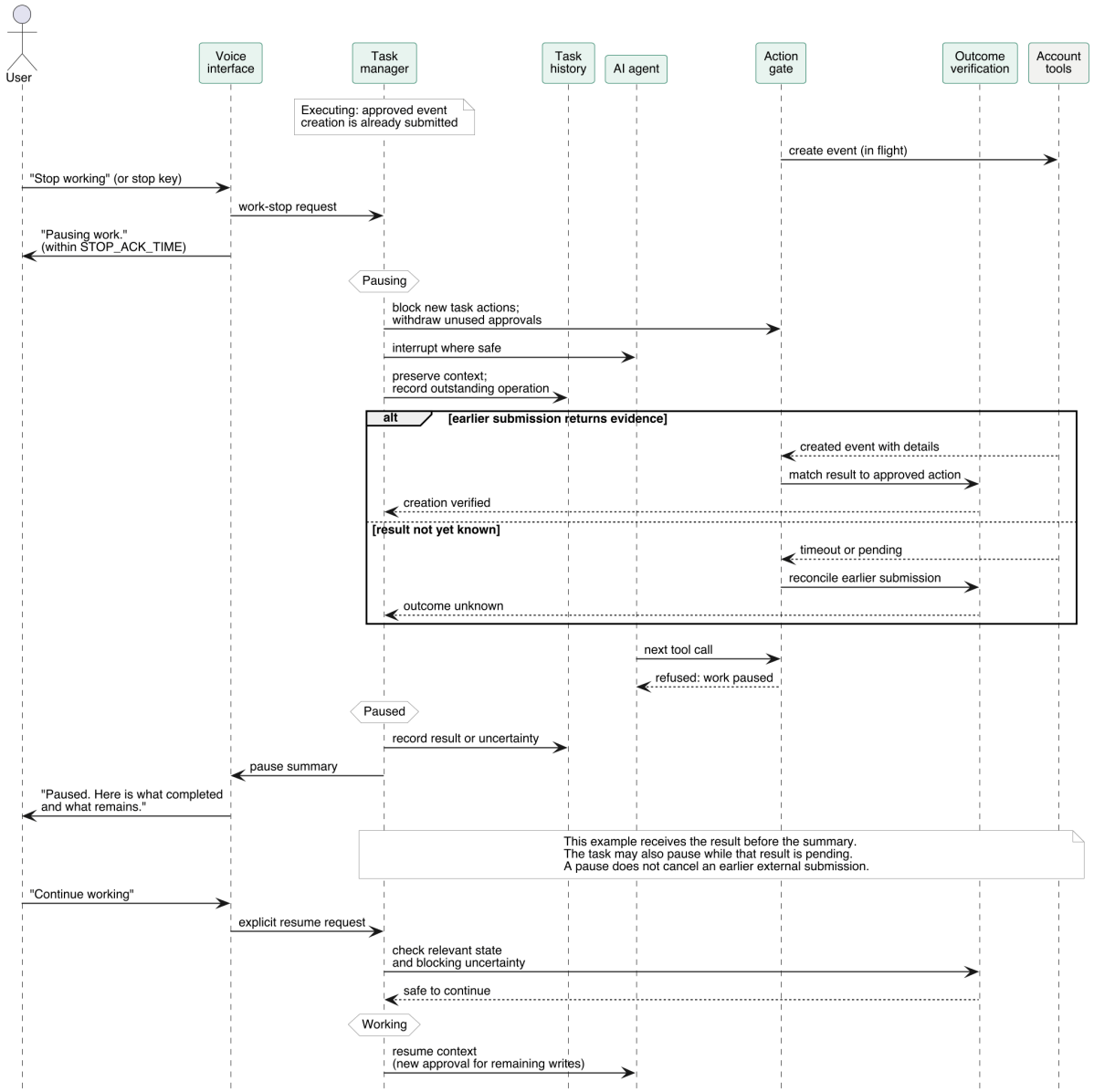


Figure 4: Pausing work while an approved action is in flight.

S.6 Detailed usage scenarios

The five use cases below expand the high-level scenarios in G.5. Each lists the main success scenario and its extensions. Extensions are numbered by the step they branch from: "2a" branches from step 2.

UC1: Reply to an email

Primary actor: User

Preconditions: A session is active, Gmail is connected through the account tools, and no other task is active.

Trigger: "Reply to Alex Chen's invitation and say Friday at 2 works for me."

Main success scenario:

1. The voice interface passes the request to the task manager. It starts the task.
2. The agent searches the inbox (read-only) and finds the invitation.
3. The agent prepares the reply without saving a draft. The gate holds the send and requests approval.
4. The system reads back the recipient, subject, and full body: "Send to Alex Chen, subject Re: Invitation: Thanks for the invitation, Friday at 2 works for me. Send it?"
5. The user says "Yes, send it."
6. The gate sends the reply with a unique action ID. Outcome verification finds the matching message in Sent.
7. The user hears "Your reply was sent to Alex Chen." The task history records the request, approval, action, and outcome.

Extensions:

- **2a.** Two possible contacts match. The system asks which one (FR4). It waits for the answer and repeats the question on request (FR5).
- **4a.** The body exceeds SHORT_MESSAGE_LENGTH. The system reads the recipient and subject. It summarizes the body and offers a full read-back (FR9).
- **4b.** The user says "change Friday to Monday." That detail changes and is read back. Earlier approval no longer applies (FR16, FR9).
- **5a.** The user declines or requests another task. Approval is withdrawn. The user hears that this reply was not sent (FR20).
- **6a.** The send cannot be confirmed. The system reports failure only with evidence of failure. Otherwise it reports unknown (FR12).

Success guarantee: Only the approved reply is sent to the intended recipient. The reported result matches the sending evidence. Delivery or reading by the recipient is not claimed.

Failure guarantee: Nothing is sent without approval of its exact recipient and text. The user hears whether sending was confirmed or remains uncertain.

UC2: Add an event from a webpage to the calendar

Primary actor: User

Preconditions: A session is active, Google Calendar is connected, and the page is public or has a saved login (FR23).

Trigger: "Add the meetup on this page to my calendar."

Main success scenario:

1. The task manager starts the task. The user hears a short acknowledgment.
2. The browser opens the page. A clear non-essential cookie prompt is rejected under FR24.
3. The agent reads the event title, date, time, and place (read-only).
4. The agent proposes event creation. The gate holds it. The system reads back the details.
5. The user approves.
6. The gate creates the event with a unique action ID. Outcome verification checks the returned details or a matching calendar record.
7. The user hears the verified result (Figure 3).

Extensions:

- **2a.** The consent prompt is unclear. The system asks the user before choosing an option (FR24).
- **3a.** The page tells the agent to ignore the user or send information elsewhere. The system treats this text as data. It does not follow those instructions (FR13).
- **3b.** The end time is missing. The system asks instead of guessing (FR4).
- **3c.** A matching event already exists. The system reports it before asking for approval.
- **5a.** The user declines. No event is created. The system reports this.

Success guarantee: The calendar contains one approved event. A matching creation response or calendar record confirms its details.

Failure guarantee: No event is created without approval. Uncertain creation is reported as unknown.

UC3: Compare information on a website

Primary actor: User

Preconditions: A session is active. The site is public, or its login was saved through the vault page (FR23).

Trigger: "On this conference site, compare the standard and student tickets."

Main success scenario:

1. The task manager starts the task.
2. The agent navigates without making a consequential change. Clear non-essential cookie prompts follow FR24.
3. The agent gathers each ticket's price and what it includes.
4. The user hears a short comparison limited to the request. Uncertain details are stated. For example, fees may be unclear (FR18, FR12).

5. The user corrects the request: "Only compare tickets that include workshops."
6. The agent rechecks and reports the corrected comparison (FR6). Nothing is purchased or submitted.

Extensions:

- **2a.** A saved login exists. The agent uses it only on its intended site (FR23).
- **2b.** No login is saved. The system explains the private setup route. The task pauses for setup.
- **2c.** The site requests a one-time code or push approval. The user may speak the code or approve on their device (FR23).
- **2d.** The site requires a device-bound passkey. The system explains the manual handoff when that requirement becomes known.
- **5a.** The user asks to buy a ticket. This requires approval of the purchase details. It follows the approval and verification flow in UC1.

Success guarantee: The comparison matches the site and the corrected request. Unconfirmed details are stated.
Failure guarantee: No purchase or submission bypasses approval. Authentication follows FR23.

UC4: Correct or pause a running task

Primary actor: User
Preconditions: A task is active.
Trigger: The user says "stop working" or presses the stop key.
Main success scenario:

1. The user requests a pause while an approved event creation is in progress.
2. The voice interface recognizes the stop phrase and says "Pausing work" within STOP_ACK_TIME (FR3, NFR3).
3. The task manager enters Pausing. The gate blocks new task actions. Unused approvals are withdrawn. Safely cancellable work is interrupted.
4. The submitted event creation cannot be cancelled safely. It returns a matching event record. The system records that result (FR8, FR12).
5. The task enters Paused. The user hears "Paused. The event was already added. The remaining work has not run."
6. The user may ask questions while work stays paused. A clear request to continue resumes the task after state checks. Any remaining writes need fresh approval.

Extensions:

- **1a.** The user corrects the task instead. The agent receives the correction (FR6). Changed action details require fresh approval (FR9).
- **1b.** The user says only "stop." Speech stops and work pauses. The system asks what the user wants next (FR3).

- **4a.** A form was prepared but not submitted. Its context is saved where possible. The system says it was not submitted. Resumption requires a check that the page still matches the request.
- **4b.** The earlier submission times out. The task remains paused. The system reports its outcome as unknown. Reconciliation may continue without advancing the task. It does not retry blindly (FR11).
- **6a.** The user cancels. The task enters Cancelling and then Cancelled. Earlier actions are reported and are not undone automatically.

Success guarantee: No new task action starts after the recognized stop request until the user resumes. Checks may only reconcile earlier actions. The user hears what completed and what remains.

Failure guarantee: An uncertain external result stays labelled unknown. A confirmed pause is not presented as cancellation of an earlier submission.

UC5: Reconnect without duplicating an action

Primary actor: User

Preconditions: A task is active and the connection drops.

Trigger: The connection to the user is lost.

Main success scenario:

1. The task pauses (FR7). The client signals connection loss within CONNECTION_NOTICE_-TIME. A local sound is used if network speech is unavailable (NFR.Sa.2).
2. The user reconnects, or starts a new session later.
3. The system briefly reports the task's state: what completed, what was pending, and anything uncertain (FR18, FR21).
4. The user says "continue." The system checks relevant state before resuming. Any unused approval is requested again.

Extensions:

- **1a.** Approval was pending. It is withdrawn. On reconnection the user hears that the pending action was not executed. Resumption needs fresh approval (FR20).
- **1b.** An action was already submitted. Its outcome is checked before a retry. The same action ID never executes twice (FR11).
- **4a.** The user cancels instead of continuing. The task ends after cancellation handling. The user hears what had already completed.

Success guarantee: No completed action is repeated as a retry. The user hears the task state before it continues.

Failure guarantee: Uncertain state is reported. Work does not resume silently.

S.7 Prioritization

- Must (POC, by November 23, 2026): FR1 to FR3, FR8, FR9, NFR3.
- Must (before participant evaluation sessions, which must complete before the March 8, 2027 V&V Report and Usability Report): FR4 to FR7, FR10 to FR16, FR18 to FR24, NFR1, NFR5, NFR.E.1 to NFR.E.3.
- Should (tune with usability evidence from those sessions, reflected in the March 8, 2027 report): conversational timing, FR17, NFR2, NFR4.

The POC shall demonstrate approval and work-stop controls for its selected email/calendar and browser workflows. This demonstrates feasibility for the tested action paths; it does not establish coverage of every integration or website action.

S.8 Verification and acceptance criteria

- Each functional requirement shall have at least one test using synthetic data and test accounts. Detailed procedures belong in the V&V Plan.
- Approval tests shall check writes and consequential commitments. No tested consequential action may start without matching approval. Tests shall also cover autonomous navigation and the FR24 exception.
- Pause tests shall cover a lookup and a submitted write. Record acknowledgment time and the time new actions are blocked. Verify that later results are reported accurately. A voice reconnection shall not resume work by itself.
- Recovery tests shall cover uncertain outcomes and stale approvals. A completed action shall not be repeated as a retry.
- Outcome tests shall use a meaningful connector response, a matching record, and a website confirmation. A queued request shall not be reported as delivered. Insufficient evidence shall produce an unknown result.
- Voice tests shall cover pauses within a sentence and short acknowledgments. They shall also cover corrections, interrupted explanations, and status requests. Record premature replies and missed interruptions. Use participant feedback to tune timing.
- Privacy checks shall verify that application history and logs exclude authentication codes. Provider processing and retention shall be reviewed separately.
- State properties P1 to P6 (S.9) shall be checked against recorded events. NFR2 shall be measured on a declared task suite. Report all attempts and failures.

S.9 Formalization: task state machine

Each task has one state $q \in Q$, where $Q = \{\text{Working, AwaitingClarification, AwaitingApproval, Executing, Pausing, Paused, Cancelling, Cancelled, Done, Failed}\}$. The partial transition function $\delta : Q \times \Sigma \to Q$ uses the events in Table 10. Figure 5 summarizes task control.

The running states are Working, AwaitingClarification, AwaitingApproval, and Executing. Pausing, Paused, and Cancelling are also active states. Cancelled, Done, and Failed are final. A session has at most one active task.

The maximum silence timer in NFR.Sa.3 counts active work in both Working and Executing, including read-only work; it excludes time awaiting user clarification or approval and time paused or cancelled.

A work-stop request blocks new task actions as soon as it is recognized. Pausing records this change and requests safe interruption. It then enters Paused. An earlier external operation may still be pending. Its outcome is tracked separately. Reconciliation checks may continue while paused. They may not advance the task. The report distinguishes unfinished work from unknown outcomes.

Cancellation uses Cancelling to block new actions and preserve earlier results. It then enters Cancelled. Earlier operations are not undone. An unresolved external submission does not justify repeating it.

Pending approvals are withdrawn on pause, disconnection, or cancellation. They do not expire merely because a connected user takes time to answer. Resumption requires a clear instruction and a check of relevant state. Unused writes need fresh approval. A new voice session alone does not resume work. New requests follow FR19.

Table 10: Task state transitions

From	Event	To
Working	required details missing or ambiguous	AwaitingClarification
AwaitingClarification	user answers	Working
Working	user corrects the task	Working
Working	consequential action or batch proposed	AwaitingApproval
AwaitingApproval	explicit approval of exact details	Executing
AwaitingApproval	user declines or changes details; pending action not run	Working
Executing	item outcome recorded; approved batch items remain	Executing
Executing	last approved item outcome recorded	Working
Working	all requested outcomes confirmed	Done
Working	no way to continue; remaining outcomes reported	Failed
any running state	work-stop request or bare “stop”; dispatch blocked	Pausing
any running state	disconnect or provider outage; dispatch blocked	Pausing
Pausing	context saved; safe interruption requested; outstanding operations recorded	Paused
Paused	status question, reconnection alone, or another stop request	Paused
Paused	clear resume instruction; relevant state checked; blocking uncertainty resolved	Working
any running state, Pausing, or Paused	explicit cancellation or confirmed task replacement	Cancelling
Cancelling	dispatch blocked; context and outstanding operations recorded	Cancelled

Required properties:

- P1: A write or consequential commitment starts only in Executing with matching approval. FR24 is the sole cookie-rejection exception.

- P2: Changed details invalidate approval. Pause, disconnection, cancellation, and task replacement withdraw all unused approvals.
- P3: A recognized work-stop request blocks new task actions until explicit resumption. Only checks of earlier outcomes may proceed. Pause feedback reports any external work still pending.
- P4: Final task states do not resume execution. Later evidence may update an earlier action's outcome in history.
- P5: Each consequential action has a recorded approval of its exact details before execution. Each approved batch item has a unique ID and executes at most once. FR24 is the stated exception.
- P6: A session has at most one active task. This includes Pausing and Cancelling.

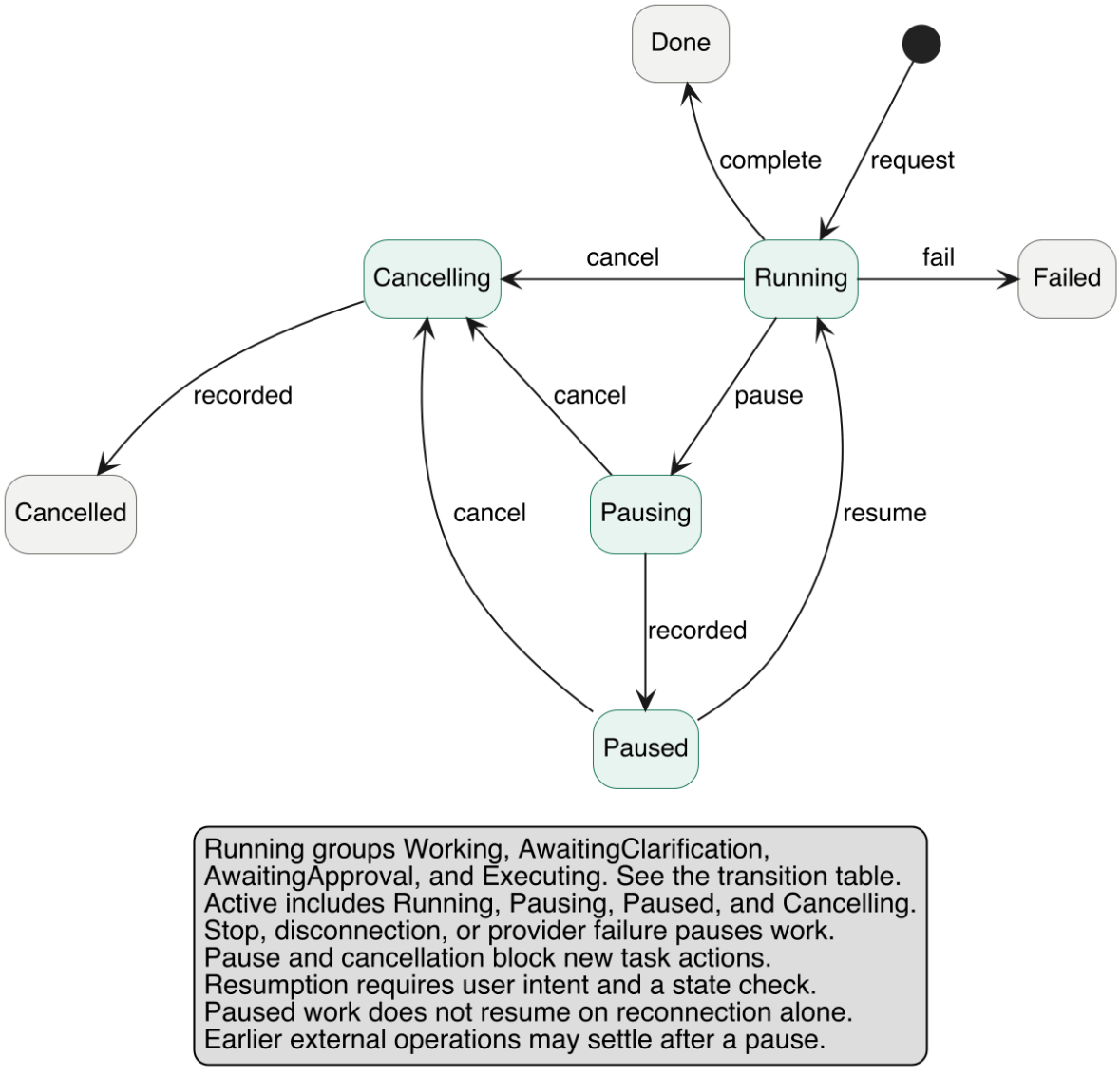


Figure 5: Task control states. Running groups the four running states in S.9. Grey states are final. External operations are tracked separately.

Project

P.1 Roles and personnel

Our team consists of four group members: Aaron Loh, Curtis Vos, Kevin Kim, and Lucas DeBoer. A dedicated teaching assistant, Rebecca Di Filippo, will meet with the team about once per month to provide guidance and progress check-ins.

As presented in Section 6 of the Development Plan, we have agreed to rotate several project management roles among all members as follows: Meeting chair, note taker, and release coordinator. Curtis is the team's external liaison for communicating with McMaster's Capstone course staff, professors, TAs, etc.

Regarding ownership/focus on certain aspects of the system's code, the following split has been agreed upon: voice and client (Lucas, reviewed by Curtis); agent and tools (Aaron, reviewed by Kevin); deployment and reliability (Kevin, reviewed by Aaron); evaluation and quality (Curtis, reviewed by Lucas). A guideline on managing conflicts or changes in development roles has been described in the aforementioned Development Plan documentation.

P.2 Imposed technical choices

- Voice and agent stack: LiveKit, Hermes, and Browser Use are all used for connecting the system's inputs, agent, and outputs.
- Languages: Python for the agent/voice backend, TypeScript and React for the client, fixed by the same Development Plan analysis.
- Target platform: Mac, Chrome, English only for the initial release, fixed by the primary evaluation group (Mac VoiceOver users).
- Connected apps and accounts: apps will be linked with Composio; app-specific API policies (testing-app caps, data-use terms) will determine how account actions must be implemented.
- Documentation and process: LaTeX in the template repository, GitHub for issues, reviews, and version control.

P.3 Schedule and milestones

- October 19, 2026: SRS and Hazard Analysis.
- November 2: V&V Plan.
- November 16: Design document revision minus one; Interview Report.
- November 23 to December 3: POC demonstration.
- January 18, 2027: Design document.
- March 8: V&V Report and Usability Report.
- March 31: Final documentation.
- April 6: EXPO.

P.4 Tasks and deliverables

Each task lists the milestone it supports and the date it must be complete. Preparatory work should be complete before the milestone it supports.

Table 11: Tasks and deliverables

Task	Description	Deliverable	Complete by
Component skeleton	Pin Hermes, LiveKit, and Browser Use versions; create placeholder modules for the voice interface, task manager, action gate, outcome verification, action tools, and task history layer (S.1).	V&V Plan	Nov 2, 2026
Deployment and CI setup	Configure GitHub Actions checks for Python and TypeScript; establish credential handling that keeps secrets out of source control and logs (NFR5).	V&V Plan	Nov 2
Voice interface integration	Wire LiveKit Agents into the task manager for audio sessions and turn handling.	Design document revision minus one; Interview Report	Nov 16
Task manager and state machine	Implement task state tracking and transitions (S.9) (FR8, FR19 to FR21).	POC demonstration	Nov 16
Hermes harness supervision	Connect Hermes on a pinned version; verify streamed status, approvals, interrupts, and reconnection.	Design document revision minus one; Interview Report	Nov 16
Participant recruitment	Recruit two to four blind participants with screen-reader experience; resolve protocol, consent, and institutional research requirements before any session (P.6).	Design document revision minus one; Interview Report	Nov 16
Action gate and approval flow	Implement the gate that requires explicit approval of exact details before any consequential action executes. Ready before the demonstration window opens.	POC demonstration	Nov 16
Browser and account tools	Integrate Browser Use and Composio as action tools behind the gate. Ready before the demonstration window opens.	POC demonstration	Nov 16
POC demonstration	Demonstrate approval and work-stop controls for the selected connect app workflows (S.7).	POC demonstration	Nov 23 - Dec 3
Issue remediation	Address problems surfaced during the POC demonstration before design revision.	Design document	Jan 18, 2027
Outcome verification	Check consequential-action results against tool responses or account/page state; record verified, partial, failed, or unknown.	Design document	Jan 18, 2027
Accessible client and task history	Build keyboard- and VoiceOver-compatible session controls and the accessible task history layer.	Design document	Jan 18, 2027

Task	Description	Deliverable	Complete by
Backend hosting configuration	Finalize the default topology, secret storage, and network requirements for the hybrid deployment once a working configuration has been demonstrated (P.5).	Design document	Jan 18, 2027
Participant evaluation integration	Incorporate interview and discovery findings.	V&V Report and Usability Report	Mar 8
Final integration and polish	Complete remaining FR/NFR coverage; finalize documentation and the known-limitations list.	Final documentation	Mar 31
EXPO demo preparation	Package and rehearse the end-to-end demonstration.	EXPO	Apr 6

P.5 Required technology elements

- LiveKit Agents (Python) and a speech-to-text/text-to-speech provider: ElevenLabs is the TTS candidate via LiveKit's plugin[**?**].
- Hermes agent and an AI model provider: Hermes is not model-specific, so a specific model is not required on its side. One candidate is currently Claude Sonnet because of its tool-call reliability under the approval-gate constraints in the Hazard Analysis, but there is flexibility to change this after POC testing or from the team's budget.
- Browser Use with Chromium.
- Connected app APIs through the Composio SDK, our own OAuth app connection before participant sessions, and test accounts.
- TypeScript and React for the web client.
- A Mac running VoiceOver for development and testing: hosted CI cannot substitute for on-device permission or accessibility checks.
- GitHub for source control and project tracking; GitHub Actions for automated Python and TypeScript checks.
- Hosting for the voice/agent backend: a hybrid of a local Mac and hosted services. A local component is required because a hosted backend cannot reach the user's Mac or its signed-in browser sessions (E.3). The default topology, secret storage, and network requirements are finalized once a working configuration is demonstrated.

P.6 Risk and mitigation analysis

- Risk: Supervision through Hermes may not work as documented.
  - Description: Hermes's documented approval, interrupt, and reconnection behavior may not match its actual behavior once integrated.
  - Mitigation: Test approvals, interrupts, and reconnection on a pinned version first; revisit the adapter/harness before promising supervision if unavailable.

- Risk: Browser actions may not be interceptable for approval.
  - Description: Browser Use may not expose a reliable point to pause and request approval before a consequential action executes.
  - Mitigation: If interception is not possible, the browser becomes read-only for the initial release.
- Risk: Recruitment of blind participants may fail.
  - Description: The team aims to recruit two to four blind participants with screen-reader experience; recruitment is not yet complete.
  - Mitigation: Report technical feasibility only and disclose missing usability evidence.
- Risk: Provider costs may exceed the budget.
  - Description: Cloud AI, speech, and hosting costs scale with usage and are not fully known in advance.
  - Mitigation: Monitor usage and reconcile monthly.
- Risk: Tasks may take longer than expected, or workload may become unbalanced across the team.
  - Description: Our four members aim to cover distinct workstreams (voice/client, agent/tools, deployment/reliability, evaluation/quality) during development; scheduling difficulties in one area can cascade into others.
  - Mitigation: Use the GitHub Kanban board to track tasks and milestones; check-in and redistribute blocked work at the weekly sync.

P.7 Requirements process and report

Requirements come from the Problem Statement, Development Plan, Hazard Analysis, TA guidance, prior research, and interviews. An interview with a former DBCS staff member was held on October 17, 2026 (transcript in the appendix); FR16 to FR18 as well as intervenors as stakeholders came as relevant additions from it. Blind participants will be recruited for repeated feedback. Requirements will be revised as POC results and interview findings arrive.

Requirement dependencies

Table 12 shows how the system requirements support the five project goals defined in the Problem Statement.

Table 12: Goal-to-requirement traceability matrix

Goal	Functional	Non-functional	Linked safety requirements
G1	FR1, FR10	NFR2	FR.Sa.13
G2	FR2, FR3, FR5, FR7, FR8, FR9, FR10, FR15, FR17, FR18, FR19, FR20, FR21, FR22, FR23, FR24	NFR1, NFR3, NFR4	FR.Sa.1, FR.Sa.2, FR.Sa.3, FR.Sa.4, FR.Sa.5.i, FR.Sa.5.ii, FR.Sa.5.iii, FR.Sa.7, FR.Sa.8, FR.Sa.11, FR.Sa.13, FR.Sa.14, FR.Sa.15, FR.Sa.17, FR.Sa.19, FR.Sa.20, NFR.Sa.1, NFR.Sa.2, NFR.Sa.3
G3	FR4, FR9, FR11, FR12, FR13, FR20, FR24	—	FR.Sa.3, FR.Sa.6, FR.Sa.9, FR.Sa.10, FR.Sa.13, FR.Sa.14, FR.Sa.15, FR.Sa.16, FR.Sa.18
G4	FR6, FR7, FR8, FR16, FR21	—	FR.Sa.5.i, FR.Sa.5.ii, FR.Sa.5.iii, FR.Sa.7, FR.Sa.8, FR.Sa.12
G5	FR14, FR22, FR23	NFR1, NFR5	FR.Sa.17

Likely and unlikely changes

Likely changes

LC1. AI model provider: not yet selected; Claude Sonnet is a candidate but Hermes is model-agnostic and can allow adjustments as needed (P.5).

LC2. Speech-to-text and text-to-speech providers: ElevenLabs is a candidate through LiveKit's plugin (P.5).

LC3. Agent harness: P.6 commits the team to revisiting the adapter if Hermes supervision does not match its documentation.

LC4. Set of connected apps: Gmail and Google Calendar are intended to be the first two integrations, not a limit on eventual scope.

LC5. Backend topology, secret storage, and network requirements: finalized once a working configuration has been demonstrated (E.3, P.5).

LC6. Browser action coverage: the browser becomes read-only for the initial release if consequential browser actions cannot be intercepted for approval (P.6).

Unlikely changes These carry the system's safety case and its reason for existing.

ULC1. Approval of exact details before a consequential action. This is the central safety property (property P1 in S.9, FR9, FR.Sa.13) and a mitigation the Hazard Analysis relies on for H5-a. Without it the system is an unsupervised agent acting for a user who cannot see what it did.

ULC2. Voice as the primary interaction mode. G.1 defines the system as a voice interface and G.7 selects the primary stakeholder group on that basis; replacing it would not be a revision of this project.

ULC3. Reporting outcomes only against matching evidence. A blind user cannot visually confirm a result, so an unverified success claim is precisely the harm that hazards H4-b and H6-a exist to prevent (FR12, FR.Sa.10).

References

Appendix: Interview transcript

Interview with a former CNIB Deafblind Community Services (DBCS) staff member.

Date: October 17, 2026
Interviewer: Aaron Loh (Group 7, Natural Control)
Participant: P1, a former special events coordinator at Deafblind Community Services (DBCS), a CNIB organization. P1 is not blind or deaf; they speak from working with DBCS clients.
Format: Conversation transcribed by Aaron.
Editing: Lightly edited for readability. Filler words, false starts, and repeated phrases were removed, and long questions were shortened to the question actually asked. The participant's answers were not reworded beyond that, and nothing was added. Square brackets mark editor notes.
Anonymity: Published without the participant's name. Their name is withheld until they confirm they are comfortable being named.

Aaron: Thank you for agreeing to do this short interview for our capstone project. To start with some context: what was your role at DBCS, and who were the people you supported? Were they fully blind, low vision, or deafblind, and what was the age range?

P1: I was a special events coordinator at Deafblind Community Services, which provided different kinds of events and services to people who are newly diagnosed with deafblindness. It's more often acquired, not something people are typically born with, so that population is in their later years. It can be earlier, but the people I mostly interacted with were older people in the community.

Our services were non-profit and government-funded, so they were free services for people who generally have low to no income. We provided different types of resources to help them navigate their home tech and other services in the community, and we provided intervenors so they could get access to the things they need. If they were going to the grocery store, an intervenor would come and assist them and provide any kind of visual context, ASL [transcribed as "ALS"; confirm], or hand-touch language.

Aaron: What tools do they use that you know of, such as VoiceOver, JAWS, NVDA, or Be My Eyes? From what you could tell, what were the limitations of those tools?

P1: For any type of screen reader, I think in that population, who already have some struggles with high-tech devices, it can be complicated. It should be oversimplified so they're able to use it. Some people probably still struggle with it, not only with using the software or having it work, but with navigating it. Also, I don't think many software programs are built with the deaf or the blind in mind. When I'm making something, I'm not thinking about how easy it is for a blind person to navigate.

Aaron: Are there any specific times you can recall seeing someone get stuck?

P1: No.

Aaron: If someone could just say what they wanted done, which tasks would matter most: email, calendar, forms, booking appointments, shopping, banking? This could be for the people you worked with, or even for you.

P1: All of the above. Scheduling, formulating emails. Those are everyday things, and they're the kinds of things that population typically looks for help with. Making appointments, they might typically need help with, like having a friend do it.

Aaron: What kind of tasks do you think people would not want AI to do for them?

P1: I'm sure there are confidentiality concerns. If there are certain healthcare problems, maybe they don't always want to disclose that to the AI. Maybe they just want to book an appointment rather than explain what is wrong. Banking could also be confidential, if they don't want to put all their banking information into an AI and ask it to move money.

Aaron: Imagine you're only talking to this assistant by voice, not looking at anything. Before it submits, deletes, or buys something, what would you want to hear first? Would you want it to confirm before it does anything?

P1: I'm neither deaf nor blind, so it's hard to say specifically. When I'm driving and can't look at what I'm typing, having Siri read back what I want to send is nice. The only downfall is that if it messes up, you have to cancel the whole message. It would be cool to have it maybe read back sentence by sentence. Confirmation that it heard you correctly.

Aaron: How much spoken feedback would you want? Say you asked it to send a message, put something on your calendar, or submit a grocery order. Would you want progress updates, or would you rather it just do it without saying much?

P1: I don't think I'd be good at answering that. I can't answer that.

Aaron: Fair. That might be better to ask someone who is actually blind, so I may ask you to connect me with someone.

Aaron: If it made a mistake, how would someone want to find out and fix it?

P1: Would it tell me that a mistake was made, or would I tell it?

Aaron: [Attribution uncertain in the notes.] Sometimes it might do something and then realize it messed up.

P1: It should tell you, I'm assuming.

Aaron: Sometimes. That question isn't working well, so let's skip it. [Question skipped.]

Aaron: Who usually sets up new technology for the people you worked with: family, or staff like you?

P1: Typically family, but there are also intervenors. Sometimes clients will dedicate some of their time with an intervenor to setting things up.

Aaron: How much time would someone spend with an intervenor?

P1: It's typically an hour to an hour and a half per day, but it varies. For some people it's a weekly occurrence at least. It's up to the client.

Aaron: Where do people typically use their computers: at home, the library, a cafe? Is there any difference for them?

P1: If anything, it might be an older device rather than a new MacBook, but I would say more at home, for comfort. It'll usually be a quieter, private place.

Aaron: Is there anything I haven't asked that you think matters for building an AI personal assistant like this for blind users?

P1: Probably just keeping it as simple as possible, because most of the time these software programs are hard to navigate, and people will skip over things that are complicated to use [transcribed as "We'll skip over"; confirm]. Sometimes, if the screen reader is reading the whole screen, it's reading irrelevant things rather than the things that actually need to be read.

Aaron: [Attribution uncertain in the notes.] So what I'm hearing is that it should read the things that matter, and only those, the things that actually have to do with getting the task done.

P1: Detailed but simple, if that makes sense.

Aaron: Detailed in what ways?

P1: Only in the ways that matter.

Aaron: Could you introduce us to two or three people who would be willing to try what we build and give feedback?

P1: Yeah, probably.

Aaron: We'll be in touch about that. Thank you for your time.

Questions not covered:

- Amount of spoken feedback: participant deferred to blind users.
- Mistake recovery: question skipped.
- Whether a voice-only tool works for deafblind users: not asked.

Appendix: AI use disclosure

Aaron Loh: Claude Code (Claude Opus 5.5) was used to work through the requirements. It asked questions, summarized tool documentation, and translated the team's decisions and Kevin's requirements draft into a structured draft. Aaron made and reviewed every decision. The team later reviewed, verified, and rewrote the draft as necessary before submission.

Kevin Kim: OpenAI Codex assisted with the October 17 layout and consistency review. It also helped revise the supervision requirements and simplify the System section from Aaron's decisions.

Curtis Vos: Claude Code (Claude Sonnet 5) was used alongside a manual review, helping with wording choices and cross-checking/verifying claims alongside other sections of the documents. All responses and suggestions were fully read through and followed up with concerns or clarifications before implementing any feedback. At the end of the review session, Claude Opus 5 was used for a more in-depth scan to verify correctness and appropriateness of section contents. If any inconsistencies (typically wording or description differences) were found, they were manually verified, compared, and corrected.

Appendix: Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Kevin Kim

1. **What went well while writing this deliverable?**

   One thing that went well was how our team divided responsibilities. Aaron and I focused on the System section, while other members worked on areas such as the goals and Hazard Analysis. This made the workload more manageable. Using AI tools to review our work also helped identify inconsistencies and missing requirements. I focused on making the requirements realistic and feasible, which helped improve the overall consistency of the SRS.

2. **What pain points did you experience, and how did you resolve them?**

   The main challenge was resolving inconsistencies between the SRS and Hazard Analysis. Some requirements described different system behaviors, so I compared both documents and helped determine which approaches were more feasible and appropriate. Another challenge was ensuring the requirements were specific enough to implement and test. Reviewing requirement traceability and refining overly broad requirements helped us address these issues.

3. **How many requirements were inspired by speaking to clients or their proxies?**

   I did not personally participate in stakeholder interviews while writing the SRS. Most of the requirements I worked on came from team discussions, where we considered the needs of blind users, along with our project goals and technical constraints. I cannot confidently identify an exact number of requirements directly inspired by stakeholder interviews. Moving forward, gathering feedback from blind users will help us validate our assumptions and improve our requirements.

4. **Which courses will help your team succeed?**

   Software Requirements, Human-Computer Interfaces, and Software Testing are three courses that I think will be especially useful. Software Requirements helped us develop and organize the SRS. Human-Computer Interfaces will help us design an accessible voice interface for blind users, while Software Testing will help us verify that the system meets its requirements, particularly for safety features such as approvals, interruptions, and error handling.

5. **What knowledge and skills will the team need to acquire?**

   Our team will need to improve our knowledge of AI agents, backend development, security, voice interfaces, and accessibility. Based on our Development Plan, I will focus on deployment, reliability, and secure backend systems. Aaron will need to develop his knowledge of AI agent frameworks and tool integration. Curtis will focus on accessibility testing and evaluation, while Lucas will need experience with voice interfaces and VoiceOver compatibility. We will also need to share knowledge to integrate these components successfully.

6. **How will each team member acquire these skills?**

   Two useful approaches are studying documentation and tutorials, and building practical prototypes. I prefer learning by doing, so I plan to experiment with backend services and error handling while using documentation and AI tools for guidance. Aaron can prototype AI agent integrations, Curtis can review accessibility guidelines and apply them through testing, and Lucas can build and test voice interactions with VoiceOver. Hands-on development will help us identify problems and learn from experience.

Aaron Loh

1. **What went well while writing this deliverable?**

   Working through the requirements really helped me and the team clarify what we are actually building. It really forced us to think through the actual use cases of how it might be used and the flows that users would go through. We were also able to work really well as a team on dividing different things, going through the important parts of our requirements together, and coming up with a shared vision for everything before we went our separate ways.

2. **What pain points did you experience and how did you resolve them?**

   One of the hardest things was defining our scope and really keeping it consistent across both the Hazard Analysis and the SRS. For example we really wanted to be ambitious with what we're doing but we also needed to keep it scoped so we definitely had some discussions there.

   Another part that was a little bit difficult was defining what stopping work means. We had to really think through what should happen in different situations to make that clear in our requirements.

3. **How many requirements were inspired by speaking to clients or their proxies?**

   I had the chance to interview a former CNIB Deafblind Community Services staff member. Their perspective on their experience and their answers to the various questions that I had were very helpful in understanding our users better, or at least a certain subset of our users. Soon we'll be planning to go through some of these things with actual blind users.

4. **Which courses will help your team succeed?**

   Software Engineering 3RA3 will definitely be a big one. Human-Computer Interfaces, which I'm taking this semester, will be really helpful as well in thinking about how people are going to be interacting and interfacing with this. It is very different from any traditional interface so we will certainly be thinking from first principles.

   Software Design will also be an important one. The things we learned in 3A04 will help because there's certainly going to be good design that needs to happen here to make sure our system works well and is easy to build on top of.

5. **What knowledge and skills will you need to acquire?**

   I think one of the biggest learnings and pieces of knowledge that I'll need to gain is in the agent ecosystem: specifically understanding the latest and greatest of how people are building these things but also how they're combining them together and designing these systems well.

   Understanding how our software can interact with the external world in a way that's flexible and truly agentic is definitely going to be an interesting topic to understand better. Designing that in a way that's secure and safe is also going to be something we're going to have to learn a lot about.

6. **How will you acquire this knowledge and these skills?**

   I would say it's going to be a lot of prototyping as well as reading through documentation, explaining what I've learned, and seeing how it applies to our own problem. Then by prototyping and really diagramming the systems I'll be able to understand them.

Lucas DeBoer

1. **What went well while writing this deliverable?**

   Like the Hazard Analysis, I found that we divided the work much better for this deliverable than the previous deliverable. We created a rough draft of the document as a team and then decided who was responsible for adding more detail to each section. As I did the Hazard analysis, I was only responsible for helping proofread the SRS document. I felt that this was a very manageable workload.

2. **What pain points did you experience, and how did you resolve them?**

   Similar to the Hazard Analysis, I felt that our biggest issue was lack of time. Once again, this was a self imposed pain point that could easily be resovled for the next deliverable. As I was responsible for the Hazard Analysis, many of the sections in the SRS counted on the safety requirements that I came up with. However, the rest of the team needed to start their sections before I could finish my section, so I need to go through and update their work once I was finished. I tried to resolve this issue as best as possible by communicating what changes I was making from the rough draft we had agreed on directly to my colleagues.

3. **How many requirements were inspired by speaking to clients or their proxies?**

   I would estimate that about 25

4. **Which courses will help your team succeed?**

   Software Requirements, Human-Computer Interfaces, and Software Testing are three courses that I think will be especially useful. Software Requirements helped us develop and organize the SRS. Human-Computer Interfaces will help us design an accessible voice interface for blind users, while Software Testing will help us verify that the system meets its requirements, particularly for safety features such as approvals, interruptions, and error handling.

   Like my teammates have mentioned. I think that Software Requirements, Human-Computer Interfaces, and Software Testing are the three most useful courses. These are directly applicable to the work that we are doing. However, I also think that our discrete math courses will be useful in future stages of the project. When we get to actually developing the project, I think the logical mindset might come in handy for figuring out some problems.

5. **What knowledge and skills will the team need to acquire?**

We will need to improve our knowledge of AI agents, security, voice interfaces, and accessibility. We have a general understanding of many of these fields, but we will need to become much more technically versed in these areas. I will be specializing in the voice interfaces and VoiceOver compatibility, so that is the areas I will focus most on.

6. **How will each team member acquire these skills?**

   I believe that reading documentation or watching tutorials of the tools we plan to use will be very valuable. As others, have mentioned hands-on prototyping will be extremely useful. Lastly, I plan on looking at existing tools that have a similar purpose to see how they work. As we have all written this section, I don't think I need to write out the best way for the others to learn.

Curtis Vos

1. **What went well while writing this deliverable?**

   The group felt quite coordinated during this deliverable, which helped the effectiveness a lot. We made sure to stay up-to-date with each other's progress through Discord by sending progress updates throughout, and our branch and PR collaboration in Github felt smooth as well.

2. **What pain points did you experience, and how did you resolve them?**

   A major pain point (as experienced when working on requirements documents in the past, like in 3RA3) felt like the ability to keep all the requirements and goals aligned with each other. When it comes to long documents like an SRS, the sheer number of goals, functional and non-functional requirements, and hazards feels very overwhelming and difficult to keep track of. This can also be mixed when it comes to group projects as well, as it's easier in terms of workload balancing when everyone helps out with different sections, but it also adds complexity in terms of making sure what I say is precisely consistent with the other sections that have been created. Making sure to read over existing work multiple times was essential, and though it can't be relied on, having AI tools scan over the documents as an extra source to verify consistency was helpful for extra reassurance.

3. **How many requirements were inspired by speaking to clients or their proxies?**

   In terms of truly creating requirements directly from our clients/proxies, it didn't feel like it was as many as I'd hoped. Based on the timing of our course and progress, we were only able to get one official interview (done by groupmate Aaron) under our belt by the time of writing the SRS, which was still very helpful for gaining valuable insight into the experiences of the visually impaired with technology. Several insights were able to arise from this interview, such as evidence regarding frequently starting tasks over with existing tools, or confirming the existing importance of people needing to intervene in existing tools, their setup, and their maintenance. However, due the fact that the interviewee was sighted, the results were not able to be too personal or based on experience. Because of this limitation, we are hoping to gain more connections in the next few weeks with the visually impaired community based off this first interaction, and refine our work based on further interactions.

4. **Which courses will help your team succeed?**

   Currently I'm taking SFWRENG 4AL3: Applications of ML, which should help a lot with understanding the inner workings of how AI can be tuned and interacted with, especially when we reach topics regarding Natural Language Processing and LLM's. I'm also taking 4HC3: Human Computer Interfaces, which seems to give great exposure to a lot of design principles, which can hopefully be applicable in making our system easily usable, especially

since our capstone project focuses so heavily on accessibility. I also think I can use a lot of knowledge from 3S03: Software Testing for the evaluation and quality control portion of development.

5. **What knowledge and skills will the team need to acquire?**

   One piece of knowledge that I think should be necessary for us to acquire is the specific tools we have settled upon for our product. Not all of us have had experience with AI harnesses like Hermes, for example, so ensuring that we are not only familiar but honestly comfortable with how AI harness tools, tools like LiveKit, etc. work will be essential for us all actually being able to contribute to a functioning system in the end. We also will need to feel comfortable with using APIs as there are multiple (e.g. for Google services in our POC) that we intend to interact with and will need to properly set up and test with.

6. **How will each team member acquire these skills?**

   AI harness tools like Hermes are a relatively new concept in the world of software, but there still appears to be recent tutorials on YouTube for getting started. Nous Research also has official documentation, which will be a necessity in development. For a solid intro, however, I will most likely find a crash course on YouTube, as I typically can feel engaged and interested in how content gets laid out for personally new concepts like this. It appears our interaction tools, like LiveKit, also have extensive tutorials and documentation, so I will likely also seek out a crash course for familiarizing myself with that specifically. For getting re-familiarized with API's, I will likely just read the necessary documentation for each area as I've had experience at my co-op and some personal projects already.